If no rescue clause matches or if an exception is raised outside a begin/end block, Ruby moves up the stack and looks for an exception handler in the caller, then in the caller's caller, and so on. If nothing catches the exception, the program typically halts.

Although the parameters to the rescue clause are typically the names of exception classes, they can be arbitrary expressions (including method calls) that return an Exception class.

Tidying Up

Sometimes you need to guarantee that particular processing is done at the end of a block of code, regardless of whether an exception was raised. For example, you may have a file open on entry to the block, and you need to make sure it always gets closed as the block exits.

The ensure clause does just this. An ensure clause goes after the last rescue clause and contains a chunk of code that will always be executed as the block terminates. It doesn't matter if the block exits normally, if it raises and rescues an exception, or if it is terminated by an uncaught exception—the ensure block will get run:

```
f = File.open("testfile")
begin
  # .. process
rescue
  # .. handle error
ensure
  f.close
end
```

You might assume that the File.open call should be inside the begin block. In this case, having the File.open inside this begin block would be a problem, because open can itself raise an exception. If the exception happened on open, you wouldn't want to run the code in the ensure block, because there'd be no file to close.

Also, in the specific case of File.open, you can pass the call a block argument which uses exception handling techniques to ensure the file is closed at the end of the block, we talked about this in Using Blocks for Transactions, on page 72:

```
File.open("testfile") do |f|
  # .. process
end
```

The else clause is a similar, although less useful, construct. If present, it goes after the rescue clauses and before any ensure. The body of an else clause is executed only if no exceptions are raised by the main body of code.

```
f = File.open("testfile")
begin
  # .. process
rescue
  # .. handle error
else
  puts "Congratulations-- no errors!"
ensure
  f.close
end
```